Exp No:1.a
Analyze the trend of data science job postings over the last decade

Description: Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

Code:

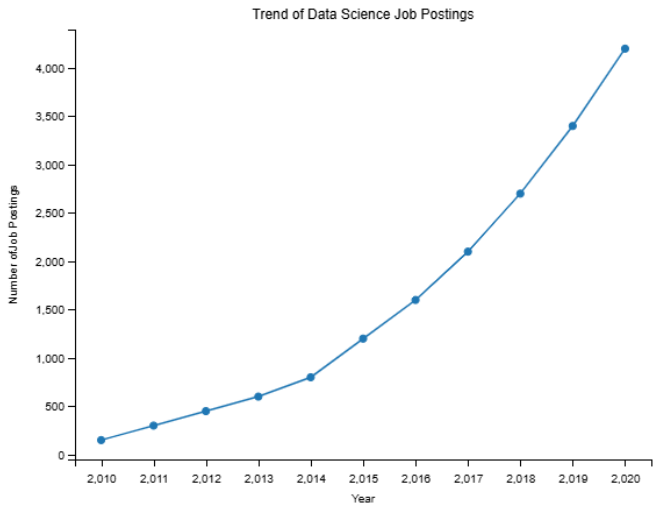
```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}
df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```

Input:

Year =2010, 2021

Job Postings=150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200

Output:



Result:

Thus, the trend of data science job postings over the last decade was successfully analyzed and visualized using Pandas and Matplotlib

Exp No:1.b

Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

Description: Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',  'Business Analyst']
counts = [300, 500, 450, 200, 150]
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```

Code:

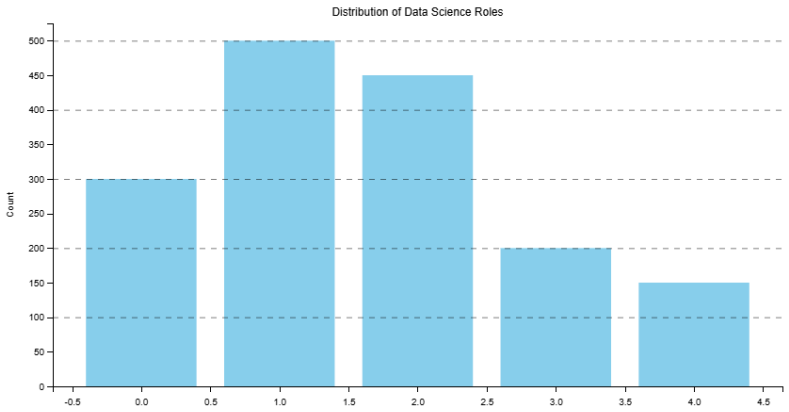
```
import pandas as pd
import matplotlib.pyplot as plt

roles = [
    'Data Analyst',
    'Data Engineer',
    'Data Scientist',
    'ML Engineer',
    'Business Analyst'
]
counts = [300, 500, 450, 200, 150]
data = {
    'Role': roles,
    'Count': counts
}
df = pd.DataFrame(data)
print(df)
plt.figure(figsize=(10, 5))
plt.bar(df['Role'], df['Count'], color='skyblue')
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.xticks(rotation=20)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()
```

Input:

Roles = Data Analyst, Data Engineer, Data Scientist, ML Engineer, Business Analyst.
Counts = 300, 500, 450, 200, 150.

Output:



Result:
Thus, the distribution of various data science roles was successfully analyzed and visualized using Python and Matplotlib.

Exp No:1.c

Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given.

Description:

Create small datasets for each type and explain their characteristics.

Code:

```
import pandas as pd

# Structured data example
structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})

print("Structured Data:")
print(structured_data)

# Unstructured data example
unstructured_data = (
    "This is an example of unstructured data. "
    "It can be a piece of text, an image, or a video file."
)

print("\nUnstructured Data:")
print(unstructured_data)

# Semi-structured data example (JSON-like dictionary)
semi_structured_data = {
    'ID': 1,
    'Name': 'Alice',
    'Attributes': {
        'Height': 165,
        'Weight': 68
    }
}

print("\nSemi-structured Data:")
print(semi_structured_data)
```

Output:

```
Structured Data:
   ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
This is an example of unstructured data. It can be a piece of text, an image, or a video file.

Semi-structured Data:
{'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

Result:
Thus, Structured, Unstructured, and Semi-Structured data were successfully differentiated and their characteristics were analyzed using Python

Exp No:1.d
Conduct an experiment to encrypt and decrypt given sensitive data.

Description:

Use the cryptography library to encrypt and decrypt a piece of data.

Aim:

To encrypt and decrypt sensitive data using the cryptography library and Fernet encryption.

Algorithm:

1. Import Fernet from the cryptography.fernet module.
2. Generate a secret encryption key using Fernet.generate_key().
3. Create a Fernet cipher object using the generated key.
4. Convert the original text into bytes.
5. Encrypt the data using encrypt().
6. Decrypt the encrypted data using decrypt().
7. Display the original, encrypted, and decrypted data.

Code:

```
from cryptography.fernet import Fernet

# Generate key

key = Fernet.generate_key()

cipher_suite = Fernet(key)

# Original data

plain_text = b"Rajalakshmi Engineering College."

# Encrypt data

cipher_text = cipher_suite.encrypt(plain_text)x

# Decrypt data

decrypted_text = cipher_suite.decrypt(cipher_text)
```

```
print("Original Data:", plain_text)

print("Encrypted Data:", cipher_text)

print("Decrypted Data:", decrypted_text)
```

Output:

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqgKBLLRhRPAfBstaK2IyURzoXdbasbheWk1RzPr5g4gxjImiMQXZc5JxXpZcbhXKoBWIsJlLpc8F33cTTYhWOVTFMzmqtajRg_RN
XSJNyZ0Oryvt_nL0I89aDkyrIwZYfzLBz'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Result:

Thus, the given sensitive data was successfully encrypted and decrypted using Fernet symmetric encryption.